NEEDLE

— permission-aware hybrid retrieval in IBM Db2 (vulnerability triage)

Team NEEDLE · 9 November 2026

> NEEDLE takes a vague security advisory and finds the services that it affects. One Db2 procedure fuses keyword and vector ranking. Db2 row permissions hide embargoed advisories from users without clearance.

01 Problem

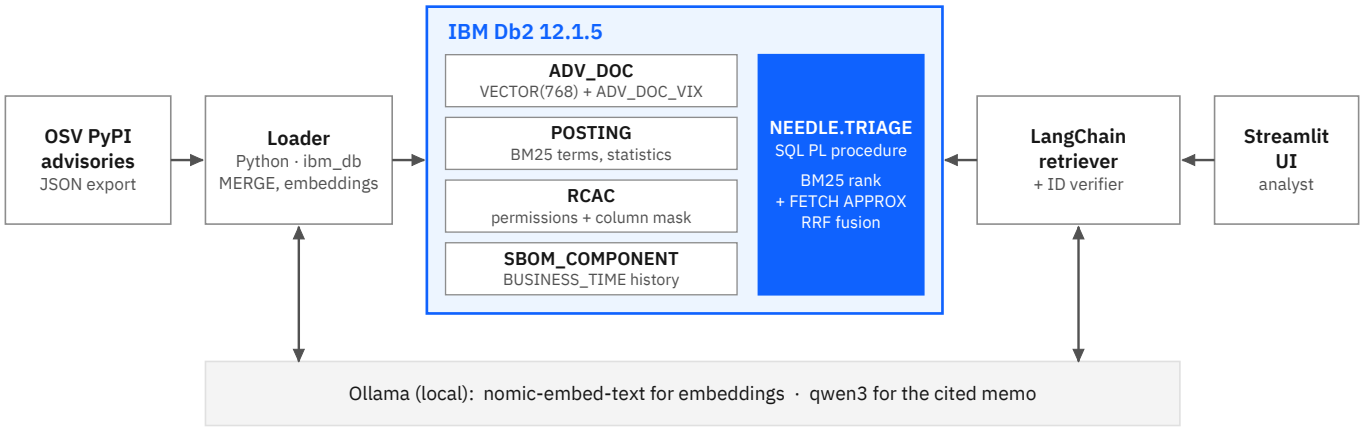
A vague advisory arrives: *"RCE in that YAML loader, no CVE yet."* It names no package, no version range and no ID.

- The security team must find the matching advisories by meaning and by identifier at once.
- The team must then find which services run an affected version, and since when.
- Some advisories are under embargo. Users without clearance must not see them in results, statistics or text.
- Scanners such as osv-scanner work after an advisory names a package and versions. NEEDLE covers the time before.

02 Solution

1. **Ingest.** A Python loader reads real OSV PyPI advisories. It keeps the raw JSON, loads rows with `MERGE`, and stores a 768-dim embedding in a `NOT NULL VECTOR(768, FLOAT32)` column with a DiskANN vector index.
2. **Retrieve.** One SQL PL procedure, `NEEDLE.TRIAGE`, fuses BM25 keyword rank (plain SQL over a postings table) with vector rank (`FETCH APPROX`) by reciprocal rank fusion (RRF). The fusion runs inside Db2.
3. **Protect.** Row and column access control (RCAC) applies to `ADV_DOC` and `POSTING`. A column mask hides embargoed details. The retriever connects as the user, so Db2 enforces visibility, not the app.
4. **Decide.** A deterministic SQL join matches candidate advisories against SBOM history in an application-period temporal table. The join returns the affected services and an "exposed since" date. No model makes this decision.
5. **Explain.** A local LLM (Ollama, qwen3) writes a cited memo through a LangChain retriever. A verifier removes any advisory, package or service ID that is not in the SQL results.

03 Architecture



Data flows left to right into Db2. Queries flow right to left: the UI calls the retriever, and the retriever calls `NEEDLE.TRIAGE` on the user's own connection, so RCAC applies to every call.

04 What we measure

Headline question: what does an RCAC row filter do to vector-index recall and to the result count as user visibility drops?

Measured (9 Nov 2026; Db2 12.1.5.0; 26,210 real OSV PyPI advisories; 20 queries; k = 10):

- `FETCH APPROX FIRST 10 ROWS ONLY` uses the vector index (EXPLAIN: IXSCAN on `ADV_DOC_VIX`). Median latency is 3.0 ms. `FETCH EXACT` scans the table in 71.7 ms. With a bound query vector: 4.8 ms and 72.3 ms.
- The APPROX and EXACT top-10 lists overlap 0.995 on average (minimum 0.90).
- `FETCH APPROX FIRST ? ROWS` (k as a parameter marker) does not use the index. NEEDLE writes k as a validated integer literal.

A plain row filter (`WHERE ROW_BUCKET < b`, not RCAC yet) lowers recall while the index stays in the plan:

Rows visible (approx.)	APPROX plan	Recall@10 (mean)	Fewest rows returned (of 10)
100%	vector index	0.995	10
50%	vector index	0.905	5
25%	vector index	0.840	5
10%	vector index	0.750	5
5%	table scan (exact)	1.000	10

In progress (no results yet):

- The same sweep under RCAC `CREATE PERMISSION`, run as users with 10% to 100% visibility. Ground truth is `FETCH EXACT` as the same user. A leak test expects zero forbidden rows.
- A plan-sharing check: does Db2 compile one plan for all users? Evidence: EXPLAIN and `MON_GET_PKG_CACHE_STMT` per user.
- A router that restores recall. It counts the rows that the user can see. It uses `FETCH EXACT` for a small visible set. Otherwise it over-fetches with a literal k from a fixed ladder, keeps the top 10, and falls back to EXACT on a short result. We fit its thresholds on a development split and report held-out queries only.
- Retrieval quality per query slice (identifier queries and vague queries) for keyword, vector and hybrid ranking.

05 Db2 features used

Feature	Role in NEEDLE
`VECTOR(768, FLOAT32)`, `VECTOR_DISTANCE`	Advisory embeddings; cosine ranking
`CREATE VECTOR INDEX`, `FETCH APPROX` / `EXACT`	DiskANN search; exact ground truth
RCAC: `CREATE PERMISSION`, `CREATE MASK`	Row visibility per user; masked embargoed details
Application-period temporal table	SBOM deploy history; "exposed since" date
SQL PL procedure	`NEEDLE.TRIAGE`: BM25 and vector fusion inside Db2
`JSON_TABLE`, `MERGE`	Shred OSV JSON; idempotent loads
`EXPLAIN`, `MON_GET_PKG_CACHE_STMT`	Plan evidence; plan sharing across users

06 Stack

IBM Db2 12.1.5 Community Edition · Python 3.13 · `ibm_db` · LangChain · Ollama (`nomic-embed-text` for embeddings, `qwen3` for the memo) · Streamlit. All models run locally.

07 Impact

- **Faster triage.** An analyst pastes a vague report. One procedure call returns candidate advisories, affected services and exposure dates.
- **Permission-safe by design.** Db2 enforces visibility on every table that carries advisory content. The LLM sees only rows that the user may see.
- **Reusable pattern for the Db2 connectors.** `langchain-db2` 1.0.0 runs exact search and filters metadata in Python after the top-k, so a filtered search can return fewer than k documents. NEEDLE keeps the filter in SQL and the vector index in the plan: a `NOT NULL` vector column, a literal k, and a visibility-aware router.

08 Status

Verified on 9 Nov 2026: the OSV load into `NEEDLE.ADV_DOC` (26,210 rows) and vector-index use (EXPLAIN). In progress: `NEEDLE.TRIAGE`, the RCAC sweep and router, the SBOM join, the retriever with the memo verifier, and the UI. We report negative results as results.